

```
1  """Annual PCU-demand scenarios for the supplied Metro Manila road dataset."""
2
3  from pathlib import Path
4
5  import numpy as np
6  import pandas as pd
7  from sklearn.compose import ColumnTransformer
8  from sklearn.ensemble import RandomForestClassifier
9  from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
10 from sklearn.pipeline import Pipeline
11 from sklearn.preprocessing import OneHotEncoder
12
13 CLASSES = ("Car", "PUJ", "UV", "Taxi", "PUB", "Truck", "Trailer", "MC", "Tricycle")
14 FEATURES = ["Road_PCU_Volume", "Capacity_Limit", "Year", "Road_Type"] + [f"Share_{c}" for c in CLASSES]
15 RISK_ORDER = ["Low", "Medium", "High"]
16 RING_ROADS = ("C-4_EDSA", "C-5_CP_Garcia")
17
18
19 def risk_from_ratio(ratio):
20     values = np.asarray(ratio, dtype=float)
21     labels = np.select([values >= 0.95, values >= 0.70], ["High", "Medium"], default="Low")
22     return str(labels.item()) if labels.ndim == 0 else labels
23
24
25 def validate_data(df):
26     required = {"Year", "Road_ID_Name", "Road_Type", "Capacity_Limit_PCU_Per_Day", "Road_PCU_Volume_Per_Day"}
27     required |= set(CLASSES) | {f"PCEF_{c}" for c in CLASSES}
28     missing = sorted(required - set(df.columns))
29     if missing:
30         raise ValueError(f"Missing columns: {missing}")
31     data = df.copy()
32     numeric = list(CLASSES) + [f"PCEF_{c}" for c in CLASSES] + ["Year", "Capacity_Limit_PCU_Per_Day", "Road_PCU_Volume_Per_Day"]
33     for col in numeric:
34         data[col] = pd.to_numeric(data[col], errors="raise")
35     if data[numeric].isna().any().any():
36         raise ValueError("Missing numeric values")
37     if data.duplicated(["Road_ID_Name", "Year"]).any():
38         raise ValueError("Duplicate road-year rows")
39     if len(data) != 120 or data.Road_ID_Name.nunique() != 20 or set(data.Year) != set(range(2020, 2026)):
40         raise ValueError("Expected 20 roads for every year 2020?2025")
41     if data.groupby("Road_ID_Name").Year.nunique().ne(6).any():
42         raise ValueError("A road is missing a historical year")
43     if (data["Capacity_Limit_PCU_Per_Day"] <= 0).any() or (data[list(CLASSES)] < 0).any().any():
44         raise ValueError("Capacity must be positive and counts nonnegative")
45     return data.sort_values(["Road_ID_Name", "Year"]).reset_index(drop=True)
46
47
48 def add_pcu_and_risk(df):
49     data = df.copy()
50     data["Road_PCU_Volume"] = sum(data[c] * data[f"PCEF_{c}"] for c in CLASSES)
51     data["Capacity_Limit"] = data["Capacity_Limit_PCU_Per_Day"]
52     data["PCU_Difference"] = data["Road_PCU_Volume"] - data["Road_PCU_Volume_Per_Day"]
53     raw_total = data[list(CLASSES)].sum(axis=1)
54     for c in CLASSES:
```

```
55     data[f"Share_{c}"] = np.where(raw_total > 0, data[c] / raw_total, 0.0)
56     data["VC_Ratio"] = data["Road_PCU_Volume"] / data["Capacity_Limit"]
57     data["Rule_Risk_Level"] = risk_from_ratio(data["VC_Ratio"])
58     return data
59
60
61 def make_features(df):
62     return df[FEATURES]
63
64
65 def fit_and_evaluate(history):
66     train = history[history.Year <= 2023]
67     test = history[history.Year >= 2024]
68     prep = ColumnTransformer([
69         ("road_type", OneHotEncoder(handle_unknown="ignore"), ["Road_Type"]),
70         ("numeric", "passthrough", [c for c in FEATURES if c != "Road_Type"]),
71     ])
72     model = Pipeline([("features", prep), ("forest", RandomForestClassifier(n_estimators=300, max_depth=8, min_samples_leaf=2, random_state=42, class_weight="balanced"))])
73     model.fit(make_features(train), train.Rule_Risk_Level)
74     pred = model.predict(make_features(test))
75     report = classification_report(test.Rule_Risk_Level, pred, labels=RISK_ORDER, output_dict=True, zero_division=0)
76     metrics = pd.DataFrame(report).T
77     matrix = pd.DataFrame(confusion_matrix(test.Rule_Risk_Level, pred, labels=RISK_ORDER), index=RISK_ORDER, columns=RISK_ORDER)
78     comparison = test[["Year", "Road_ID_Name", "Road_Type", "VC_Ratio", "Rule_Risk_Level"]].copy()
79     comparison["RF_Risk_Level"] = pred
80     comparison["Disagreement"] = comparison.Rule_Risk_Level.ne(comparison.RF_Risk_Level)
81     return model, metrics, matrix, comparison
82
83
84 def simulate(history, model, name, growth_rate=0.045, transfer_share=0.0, capacity_increase=0.0, expansion_year=2028):
85     """Transfer share moves car *people* to buses: one new bus per 40 shifted cars."""
86     if not 0 <= transfer_share <= 1 or growth_rate < -1 or capacity_increase < 0:
87         raise ValueError("Invalid scenario parameters")
88     baseline = history.loc[history.Year == 2025].copy()
89     output = []
90     for year in range(2025, 2036):
91         state = baseline.copy()
92         state["Year"] = year
93         factor = (1 + growth_rate) ** (year - 2025)
94         for c in CLASSES:
95             state[c] = baseline[c].to_numpy(dtype=float) * factor
96         if year > 2025 and transfer_share:
97             shifted_cars = state.Car * transfer_share
98             state["Car"] -= shifted_cars
99             state["PUB"] += shifted_cars / 40.0
100         state["Capacity_Limit_PCU_Per_Day"] = baseline.Capacity_Limit.to_numpy(dtype=float)
101         if year >= expansion_year and capacity_increase:
102             mask = state.Road_ID_Name.isin(RING_ROADS)
103             state.loc[mask, "Capacity_Limit_PCU_Per_Day"] *= 1 + capacity_increase
104         state = add_pcu_and_risk(state)
105         state["RF_Risk_Level"] = model.predict(make_features(state))
106         state["Risk_Level"] = state["Rule_Risk_Level"]
107         state["RF_Disagreement"] = state.RF_Risk_Level.ne(state.Risk_Level)
```

```
108         state["Scenario"] = name
109         state["Capacity_Basis"] = "Synthetic scenario, not measured"
110         output.append(state)
111         result = pd.concat(output, ignore_index=True)
112         first_high = result.loc[result.Risk_Level.eq("High")].groupby("Road_ID_Name").Year.min()
113         result["First_High_Year"] = result.Road_ID_Name.map(first_high).astype("Int64")
114         return result
115
116
117 def save_outputs(results, directory):
118     folder = Path(directory)
119     folder.mkdir(parents=True, exist_ok=True)
120     cols = ["Scenario", "Year", "Road_ID_Name", "Road_Type", "Road_PCU_Volume", "Capacity_Limit", "VC_Ratio", "Risk_Level", "RF_Risk_Level", "RF_Disagreement",
121            "First_High_Year", "Capacity_Basis"]
122     results[cols].to_csv(folder / "road_year_forecast.csv", index=False)
123     results.loc[results.Year.eq(2035), ["Scenario", "Road_ID_Name", "Road_Type", "VC_Ratio", "Risk_Level", "First_High_Year"]].to_csv(folder / "scenario_2035_summary.csv", index=False)
```

```
1  """Interactive, terminal-only runner for the Metro Manila AADT model."""
2
3  from pathlib import Path
4  import re
5  import textwrap
6
7  import pandas as pd
8
9  from aadt_model import add_pcu_and_risk, fit_and_evaluate, save_outputs, simulate, validate_data
10
11  ROOT = Path(__file__).resolve().parent
12  DATA = ROOT / "data" / "metro_manila_aadt_simulation_input_2020_2025.csv"
13
14
15  def ask_choice(prompt, choices):
16      while True:
17          answer = input(prompt).strip().lower()
18          if answer in choices:
19              return answer
20          print(f"Please enter: {' '.join(sorted(choices))}")
21
22
23  def ask_year():
24      while True:
25          answer = input("Which year would you like to see? 2025-2035 [Enter = 2035]: ").strip()
26          if not answer:
27              return 2035
28          if answer.isdigit() and 2025 <= int(answer) <= 2035:
29              return int(answer)
30          print("Enter a year from 2025 through 2035.")
31
32
33  def ask_growth():
34      while True:
35          answer = input("Yearly traffic growth percentage, 0-30 [Enter = 4.5]: ").strip()
36          try:
37              value = 4.5 if not answer else float(answer)
38          except ValueError:
39              value = -1
40          if 0 <= value <= 30:
41              return value / 100
42          print("Enter a number from 0 to 30.")
43
44
45  def choose_scenario():
46      print("\nCHOOSE A SCENARIO")
47      print("-" * 100)
48      print(f"{'No.':<5} {'Scenario':<31} {'What question does it answer?'}")
49      print("-" * 100)
50      rows = [
51          ("1", "Baseline: 4.5% yearly growth", "What happens if traffic grows at the assumed rate?"),
52          ("2", "Faster growth: 7% yearly", "What if traffic grows faster than expected?"),
53          ("3", "Transit shift: 20% of cars", "What if some car users move to buses?"),
54          ("4", "EDSA/C-5: 20% more capacity", "What if these roads gain capacity in 2028?"),
```

```

55     ("5", "Your own growth rate", "What if traffic grows at a rate you choose?"),
56     ("0", "Exit", "Close the simulation."),
57 ]
58 for number, label, goal in rows:
59     print(f"{number:<5} {label:<31} {goal}")
60 print("-" * 100)
61 choice = ask_choice("Enter a scenario number (0-5): ", set("012345"))
62 scenarios = {
63     "1": ("Baseline 4.5%", {"growth_rate": 0.045}),
64     "2": ("Accelerated 7%", {"growth_rate": 0.07}),
65     "3": ("Transit shift 20%", {"growth_rate": 0.045, "transfer_share": 0.20}),
66     "4": ("EDSA/C-5 expansion 20%", {"growth_rate": 0.045, "capacity_increase": 0.20}),
67 }
68 if choice == "5":
69     growth = ask_growth()
70     return f"Custom growth {growth * 100:g}%", {"growth_rate": growth}
71 return scenarios.get(choice)
72
73
74 def show_results(result, year):
75     frame = result.loc[result.Year.eq(year)].copy()
76     baseline = result.loc[result.Year.eq(2025), ["Road_ID_Name", "Total_Raw_Volume_Reported", "Road_PCU_Volume"]].set_index("Road_ID_Name")
77     counts = frame.Risk_Level.value_counts().reindex(["Low", "Medium", "High"], fill_value=0)
78     year_alert = "High" if counts["High"] else "Medium" if counts["Medium"] else "Low"
79     high_roads = frame.loc[frame.Risk_Level.eq("High"), "Road_ID_Name"].tolist()
80     width = 108
81
82     def border(char="-"):
83         return "+" + char * (width - 2) + "+"
84
85     def line(message=""):
86         for part in textwrap.wrap(str(message), width - 4, break_long_words=False) or [""]:
87             print("| " + part.ljust(width - 4) + " |")
88
89     def table_line(message):
90         if len(message) > width - 4:
91             raise ValueError("Result row exceeds terminal box width")
92         print("| " + message.ljust(width - 4) + " |")
93
94     print("\n" + border("="))
95     line(f"TRAFFIC RISK RESULT | {result.Scenario.iat[0]} | YEAR {year}")
96     print(border())
97     line(f"AT A GLANCE: {year} is {year_alert.upper()} ALERT for this 20-road group.")
98     line(f"In {year}: {counts['High']} roads are High, {counts['Medium']} are Medium, and {counts['Low']} are Low.")
99     line("The group alert uses the highest road risk; it does not mean every road has that risk.")
100    line("High-risk roads this year: " + (" ".join(high_roads) if high_roads else "none"))
101    print(border())
102    line(f"ROAD-BY-ROAD RISK IN {year}")
103    header = f"{'Road':<20} {'2025 vehicles':>13} {'2025 PCU':>10} {str(year) + ' PCU':>10} {'Cap. used':>9} {str(year) + ' risk':>9} {'AI risk':>8} {'High from':>9}"
104    table_line(header)
105    print(border())
106    for _, road in frame.sort_values("VC_Ratio", ascending=False).iterrows():
107        first_high = "-" if pd.isna(road.First_High_Year) else str(int(road.First_High_Year))

```

```

108         row = (f"{road.Road_ID_Name:<20} {baseline.loc[road.Road_ID_Name, 'Total_Raw_Volume_Reported']:>13,.0f} "
109                f"{baseline.loc[road.Road_ID_Name, 'Road_PCU_Volume']:>10,.0f} "
110                f"{road.Road_PCU_Volume:>10,.0f} {road.VC_Ratio:>8.1%} "
111                f"{road.Risk_Level:>9} {road.RF_Risk_Level:>8} {first_high:>9}")
112         table_line(row)
113     print(border())
114     line("WHAT THE COLUMNS MEAN")
115     line("2025 vehicles/day = reported vehicle count in the supplied dataset (source unverified).")
116     line("2025 PCU/day = that vehicle mix converted to passenger-car units; this is calculated, not measured.")
117     line(f"{year} PCU/day = projected daily traffic under the selected scenario. For 2025, this is the baseline, not a forecast.")
118     line("Capacity used = projected traffic divided by assumed road capacity; the capacity is not measured.")
119     line("Risk assessment = Low below 70%, Medium from 70% to below 95%, High at 95% or more.")
120     line("AI assessment = Random Forest label. It can disagree with the threshold-based risk assessment.")
121     line(f"The AI assessment differs from the threshold assessment for {int(frame.RF_Disagreement.sum())} roads in {year}.")
122     line("High from = first modeled year at 95% capacity or more; '-' means no crossing by 2035.")
123     line("These are scenario results for planning, not measured capacity or a prediction of physical road failure.")
124     print(border("="))
125
126
127 def main():
128     print("\nMETRO MANILA ROAD TRAFFIC SIMULATION | 2025-2035")
129     print("Goal: explore when projected daily traffic may approach or exceed road capacity on 20 roads.")
130     print("How to use: choose a scenario number, choose a year, then decide whether to save the full forecast.")
131     print("After each result, the menu returns so you can compare another scenario. Enter 0 to exit.")
132     print("The results are planning examples because the supplied road capacities are synthetic estimates.\n")
133     print("Checking data and preparing the Random Forest risk model...")
134     history = add_pcu_and_risk(validate_data(pd.read_csv(DATA)))
135     model, metrics, _, comparison = fit_and_evaluate(history)
136     print(f"Model check: the Random Forest matched {metrics.loc['accuracy', 'support']:.1%} of 2024-2025 risk labels.")
137     print("This measures risk-label matching, not accuracy of future traffic forecasts.")
138     while True:
139         scenario = choose_scenario()
140         if scenario is None:
141             print("Goodbye.")
142             return
143         name, parameters = scenario
144         result = simulate(history, model, name, **parameters)
145         year = ask_year()
146         show_results(result, year)
147         if ask_choice("Save every year of this scenario as CSV files? (y/n): ", {"y", "n"}) == "y":
148             slug = re.sub(r"^[a-z0-9]+", "_", name.lower()).strip("_")
149             destination = ROOT / "outputs" / "terminal" / slug
150             save_outputs(result, destination)
151             print(f"Saved {destination / 'road_year_forecast.csv'}")
152             print(f"Saved {destination / 'scenario_2035_summary.csv'}")
153
154
155 if __name__ == "__main__":
156     try:
157         main()
158     except (KeyboardInterrupt, EOFError):
159         print("\nStopped.")

```